ADF Pipeline Setup Guide

XML to Parquet — CIC Credit Data

Azure Data Factory | ADLS Gen2 | Mapping Data Flow

This document describes the 6 steps required to build an Azure Data Factory pipeline that reads CIC credit report XML files from Azure Data Lake Storage Gen2, parses the nested structure, and writes clean Parquet tables ready for analytics.

Overview

The pipeline reads multiple XML files from a single ADLS Gen2 folder and produces 5 normalised Parquet tables:

Parquet table	Contents
`legal_information`	Customer identity — CIC code, name, national ID
`credit_card_outstanding`	One row per credit card per customer
`outstanding_12_months`	Monthly outstanding balance history (12 months)
`inquiry_history`	Credit inquiry log — which bank, when, what product
`credit_score`	CIC score, grade, percentile ranking

> *All tables share CIC_CODE, NATIONAL_ID as a foreign key so they can be joined in downstream analytics tools such as Azure Synapse, Power BI, or Databricks.*

Step 1.1 - Create a Resource Group

Step 1.2 - Create the ADF Instance

Create an Azure Data Factory instance in the Azure portal. This is a one-time setup.

Instructions

1. Go to portal.azure.com and search for Data factories.
2. Click + Create. Fill in the required fields:

Field	Value
`Resource group`	Use the same group as your storage account (e.g. rg-cic-project)
`Name`	Any unique name, e.g. adf-cic-parser
`Region`	Closest region to your team
`Version`	V2

3. Click Review + create then Create. Deployment takes 1-2 minutes.
4. When deployment completes, click Go to resource then Launch studio.

Step 2 — Create the Storage Account (ADLS Gen2)

Create an Azure Storage Account with the Hierarchical namespace toggle enabled. This converts it from plain Blob Storage into Azure Data Lake Storage Gen2, which is required for analytics workloads.

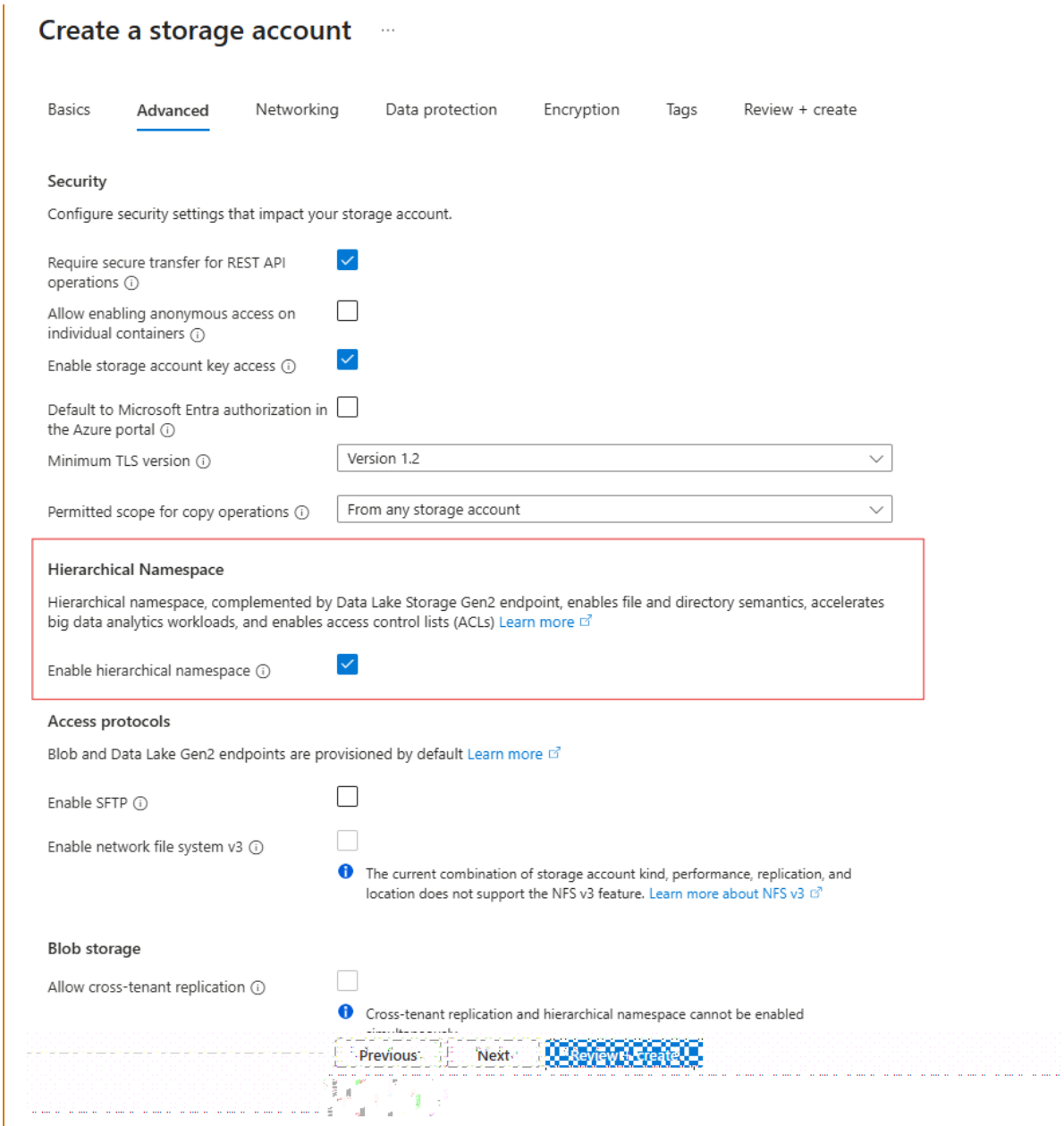
Instructions

5. In the Azure portal search for Storage accounts and click + Create.
6. On the Basics tab fill in the required fields:

Field	Value
`Resource group`	Same group as your ADF instance
`Storage account name`	Lowercase letters and numbers only, e.g. adlscicparser
`Region`	Same region as ADF to avoid data transfer costs
`Performance`	Standard
`Redundancy`	LRS (locally-redundant) is sufficient for development

7. Click the Advanced tab. Under Data Lake Storage Gen2, turn on Hierarchical namespace.

> *This toggle cannot be enabled after the account is created. Confirm it is ON before clicking Create.*



8. Click Review + create then Create.
9. After deployment, open the storage account, go to **Data Storage** => **Container**, and create two containers:

Container name	Purpose
`input`	Upload XML files here — folder path: input/cic/
`output`	ADF writes Parquet files here — one subfolder per table

10. Upload your XML files into the input/cic/ folder.

Step 3 — Create the Linked Service

A Linked Service is the connection string that tells ADF where your storage account is and how to authenticate. You only need one Linked Service — it is reused by all datasets.

Instructions

11. In ADF Studio, click Manage (toolbox icon) in the left sidebar.
12. Click Linked services then + New.
13. Search for Azure Data Lake Storage Gen2 and select it.
14. Fill in the configuration:

Field	Value
`Name`	ls_blob_source
`Authentication method`	Account key
`Storage account name`	Select your ADLS Gen2 account from the dropdown

15. Click Test connection. A green checkmark confirms ADF can reach your storage.
16. Click Create to save.

> *If Test connection fails with a permission error, go to your storage account in the Azure portal, open Access control (IAM), and assign the Storage Blob Data Contributor role to your ADF instance's managed identity.*

Step 4 — Create Datasets

Datasets define what your files look like and where they live. You need one XML source dataset and five Parquet sink datasets.

4a — XML source dataset

17. In Azure Data Factory Studio, click Author (pencil icon) in the left sidebar.
18. Click Datasets then + New dataset.
19. Search for XML and select the XML tile (not Binary, not DelimitedText).
20. Fill in the Connection tab:

Field	Value
`Name`	ds_xml_source
`Linked service`	ls_blob_source
`Container`	input
`Directory`	cic
`Row tag`	(leave empty)
`Compression`	None

21. Click Publish all or the Save icon.

4b — Parquet sink datasets (5 total)

Create one Parquet dataset per output table. For each: click + New dataset, search Parquet, select Parquet, then fill in:

Dataset name	Directory
`ds_parquet_legal`	output/legal_information/
`ds_parquet_cards`	output/credit_card_outstanding/
`ds_parquet_history`	output/outstanding_12_months/
`ds_parquet_inquiries`	output/inquiry_history/
`ds_parquet_score`	output/credit_score/

For each Parquet dataset set Linked service to your ADLS Gen2 linked service and Compression codec to snappy. On the Schema tab click Import schema and choose From connection/store to let ADF infer the schema automatically from the Data Flow output.

Step 5 — Build the Mapping Data Flow

Build one Data Flow per output table. Each follows the same four-step pattern: Source > Flatten > Select > Sink. The example below creates the credit_card_outstanding table. Repeat for the other four tables, changing the Flatten path and Sink dataset.

> *Enable Data flow debug at the top of the canvas before starting. This spins up a small Spark cluster so you can use Data preview to check each transformation as you build it.*

5a — Source transformation

22. Click Author > Data flows > + New data flow > Mapping Data Flow.
23. Name it df_credit_cards.
24. Click + Add source on the canvas.
25. In the Source settings panel at the bottom, set the Dataset field to your XML source dataset.
26. Enable Allow schema drift and Infer drifted column types on the Source options tab.
27. Click the Projection tab and click Import projection. This loads the schema into the Data Flow engine and is required before the Flatten dropdown will show any columns.
28. Click Data preview to confirm you see a CREDIT_REPORT struct column with nested sub-columns.

5b — Flatten transformation

29. Click the + on the right edge of the Source box and select Flatten.
30. In the Unroll by field click + then use the dropdown to navigate the schema tree:

```
CREDIT_REPORT > CURRENT_CREDIT_RELATIONSHIPS > CREDIT_CARD_OUTSTANDING
> RECORD
```

31. Select RECORD. Confirm it shows array<struct> next to it.
32. Set Unroll root to CREDIT_REPORT.
33. Click Data preview. You should see one row per credit card record (4 rows for a typical CIC file).

> *If the Unroll by dropdown is empty after clicking +, go back to the Source transformation and import the projection first (step 5a point 6). The dropdown only populates after the projection is loaded.*

5c — Select transformation

34. Click + on the Flatten box and select Select.
35. Add the following column mappings:

Input path	Output column name
`RECORD.STATEMENT_DATE`	STATEMENT_DATE
`RECORD.INSTITUTION_CODE`	INSTITUTION_CODE
`RECORD.INSTITUTION_NAME`	INSTITUTION_NAME
`RECORD.NUMBER_OF_CARDS`	NUMBER_OF_CARDS
`RECORD.CREDIT_LIMIT`	CREDIT_LIMIT
`RECORD.AMOUNT_DUE`	AMOUNT_DUE
`RECORD.OVERDUE_AMOUNT`	OVERDUE_AMOUNT
`RECORD.OVERDUE_DAYS`	OVERDUE_DAYS
`LEGAL_INFORMATION.CIC_CODE`	CIC_CODE (foreign key)

5d — Sink transformation

36. Click + on the Select box and select Sink.
37. Set Dataset to ds_parquet_cards.
38. On the Settings tab set:

Setting	Value
`Write behavior`	Append (preserves existing rows on repeat runs)
`File name option`	Output to single file
`Output file name`	credit_card_outstanding.parquet

Step 6 — Create the Pipeline and Run

Wrap all Data Flows in a single pipeline so they can be triggered together.

Instructions

39. Click Author > Pipelines > + New pipeline. Name it pl_xml_to_parquet.
40. From the Activities panel drag a Data flow activity onto the canvas for each of your 5 Data Flows.
41. In each activity's Settings tab set the Data flow field to the corresponding flow.

> *Click Debug at the top of the pipeline canvas. Each activity turns green when complete.*

42. Check the output container in your storage account — 5 Parquet files should appear, one in each subfolder.
43. Once debug succeeds, click Publish all to save everything permanently.
44. Optionally click Add trigger > New/Edit to schedule the pipeline or trigger it automatically when new XML files land in the input/cic/ folder using a Storage Event trigger.

Expected output structure

File path	Contents after first run
`output/legal_information/legal_information.parquet`	1 row per customer
`output/credit_card_outstanding/credit_card_outstanding.parquet`	1 row per card per customer
`output/outstanding_12_months/outstanding_12_months.parquet`	12 rows per customer (one per month)
`output/inquiry_history/inquiry_history.parquet`	1 row per inquiry event per customer
`output/credit_score/credit_score.parquet`	1 row per customer

> *On subsequent runs, new XML files dropped into input/cic/ will be appended into the existing Parquet files. Rows are never deleted. If you need to reprocess from scratch, delete the Parquet files in the output container first, then re-run the pipeline.*